

VEHICLE SERVICE CENTRE MANAGEMENT SYSTEM



Name: K. Devika

Roll No: 25WU0102120

Course: Database Management System(DBMS)

Faculty Name: Kiran Mayee

Academic Year: 2026-27

TABLE OF CONTENTS

1. Cover Page
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL, DML) with Sample Outputs
10. Queries with Outputs
11. UI Design and Screenshots
12. Implementation Details
13. Testing
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

2. Abstract

Vehicle service centres handle a continuous flow of customers, vehicles, appointments, repair tasks, mechanics, spare parts and payments. When these records are maintained manually or in scattered spreadsheets, they are prone to duplication, inconsistency and loss, and it becomes difficult to answer simple questions such as which jobs are pending, how much stock remains or how much revenue has been earned.

This project presents the design and implementation of a relational Database Management System for a Vehicle Service Centre. The system is modelled using an Entity–Relationship diagram and converted into a normalized relational schema of twelve tables: CUSTOMER, VEHICLE, SERVICE_BOOKING, JOB_CARD, TASK, MECHANIC, ASSIGNMENT, INSPECTION, SPARE_PART, PART_USAGE, INVOICE and PAYMENT. Data integrity is enforced through primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT and CHECK constraints.

The database was implemented in MySQL, populated with sample data, and validated using ten SQL queries covering selection, joins and aggregation, along with constraint-level test cases. The result is a consistent, well-structured data foundation on which a complete service-centre management application can be built. A Python Tkinter desktop interface connected to the database allows records to be viewed, inserted and deleted.

Keywords: DBMS, MySQL, Python, Tkinter, ER Diagram, Normalization, Vehicle Service Centre, SQL

3. Introduction and Problem Statement

3.1 Introduction

A vehicle service centre provides maintenance and repair services such as oil changes, brake repairs, engine tuning, AC servicing and wheel alignment. A single visit involves many connected pieces of information: who the customer is, which vehicle they brought, when the service was booked, what inspection revealed, which tasks were performed by which mechanic, which spare parts were consumed, and how the final bill was paid.

A Database Management System (DBMS) stores this information in a structured, shared and secure manner, allowing it to be queried quickly and kept consistent. The relational model, in which data is organised into tables linked by keys, is the natural choice for this domain because the data is highly structured and the relationships between entities are well defined.

3.2 Problem Statement

Many small and medium service centres still depend on paper registers or loosely maintained spreadsheets. This leads to the following problems:

- Customer and vehicle details are recorded repeatedly, causing redundancy and inconsistency.
- There is no reliable link between a booking, the work performed, the parts consumed and the final invoice.
- Mechanic workload and task history cannot be tracked easily.
- Spare-part stock and usage are not recorded systematically.
- Billing and payment records are hard to reconcile, and revenue reports take considerable manual effort.

The problem addressed by this project is to design and implement a relational database that captures the complete service workflow — from customer registration to payment — while guaranteeing data integrity and supporting useful analytical queries.

4. Objectives and Scope

4.1 Objectives

- To analyse the functional requirements of a vehicle service centre and identify its entities and relationships.
- To design an ER diagram that models customers, vehicles, bookings, job cards, tasks, mechanics, inspections, spare parts, invoices and payments.
- To convert the ER diagram into a relational schema and normalize it up to Third Normal Form (3NF).
- To implement the schema in MySQL using DDL statements with appropriate keys and integrity constraints.
- To populate the database with realistic sample data using DML statements.
- To write and verify SQL queries that retrieve, join and summarise the stored information.
- To test the database against valid and invalid inputs to confirm that constraints behave correctly.

4.2 Scope

The project covers the database and a desktop interface for the service centre system:

- Registration of customers and their vehicles.
- Service booking and job card creation (one job card per booking).
- Task management and assignment of tasks to mechanics with start and end times.
- Recording inspection findings and spare-part usage per job card.
- Invoice generation and recording of payments by Cash, Card, UPI or Online mode.
- A Python Tkinter desktop interface to view, insert and delete records in all twelve tables.

Out of scope: payroll, supplier and purchase management, customer loyalty programmes, and online payment-gateway integration. Spare-part stock is stored but is not automatically reduced when parts are used; this is listed as a future enhancement.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Specification
Operating System	Windows 10 / 11
Database	MySQL Server 8.0
Database Client	MySQL Workbench
Programming Language	Python 3.x
GUI Library	Tkinter (included with Python)
Database Connector	mysql-connector-python
Code Editor	Visual Studio Code
Diagramming / Slides	Canva
Version Control	Git and GitHub

5.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or equivalent
RAM	8 GB
Storage	At least 1 GB of free disk space
Display	1366 × 768 resolution or higher
Other	Keyboard, mouse; internet connection for GitHub access

6. ER Diagram

The Entity–Relationship diagram below models the Vehicle Service Centre. Rectangles represent entities, ellipses represent attributes (underlined attributes are primary keys), and diamonds represent relationships with their cardinalities.

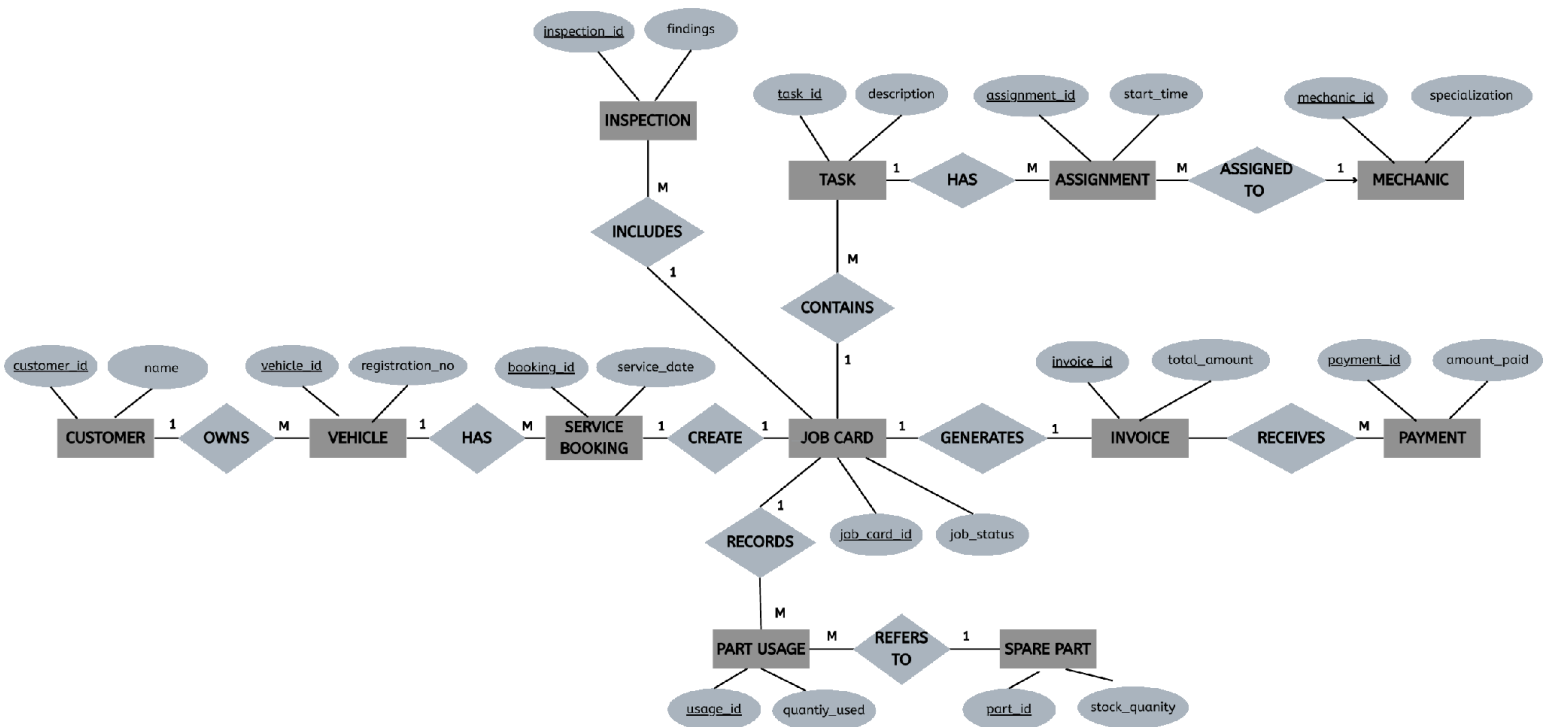


Figure 6.1: ER diagram of the Vehicle Service Centre Management System

6.1 Relationships and Cardinality

Entity A	Relationship	Cardinality	Entity B / Explanation
CUSTOMER	OWNS	1 : M	VEHICLE – a customer may own many vehicles
VEHICLE	HAS	1 : M	SERVICE_BOOKING – a vehicle can be booked many times
SERVICE_BOOKING	CREATE	1 : 1	JOB_CARD – each booking produces one job card
JOB_CARD	CONTAINS	1 : M	TASK – a job card contains many tasks
JOB_CARD	INCLUDES	1 : M	INSPECTION – a job card has inspection records
JOB_CARD	RECORDS	1 : M	PART_USAGE – parts consumed on the job
SPARE_PART	REFERS TO	1 : M	PART_USAGE – a part can be used many times
JOB_CARD	GENERATES	1 : 1	INVOICE – one invoice per job card
INVOICE	RECEIVES	1 : M	PAYMENT – an invoice may be paid in instalments
TASK	HAS	1 : M	ASSIGNMENT – a task can have assignment records
MECHANIC	ASSIGNED TO	1 : M	ASSIGNMENT – a mechanic handles many assignments

The many-to-many relationship between TASK and MECHANIC is resolved through the ASSIGNMENT entity, which also stores the start and end time of the work.

7. Relational Schema and Normalization

7.1 Relational Schema

The ER diagram was mapped to the following relational schema. Primary keys are underlined in the data dictionary and foreign keys are marked FK.

Relation	Attributes
CUSTOMER	<u>Customer_ID</u> (PK), Name, Phone, Email
VEHICLE	<u>Vehicle_ID</u> (PK), Registration_No, Model, Customer_ID (FK)
SERVICE_BOOKING	<u>Booking_ID</u> (PK), Service_Date, Status, Vehicle_ID (FK)
JOB_CARD	<u>Job_Card_ID</u> (PK), Created_Date, Job_Status, Booking_ID (FK)
TASK	<u>Task_ID</u> (PK), Description, Labour_Charge, Job_Card_ID (FK)
MECHANIC	<u>Mechanic_ID</u> (PK), Name, Specialization, Phone
ASSIGNMENT	<u>Assignment_ID</u> (PK), Start_Time, End_Time, Task_ID (FK), Mechanic_ID (FK)
INSPECTION	<u>Inspection_ID</u> (PK), Inspection_Date, Findings, Job_Card_ID (FK)
SPARE_PART	<u>Part_ID</u> (PK), Part_Name, Stock_Quantity, Unit_Price
PART_USAGE	<u>Usage_ID</u> (PK), Quantity, Job_Card_ID (FK), Part_ID (FK)
INVOICE	<u>Invoice_ID</u> (PK), Invoice_Date, Total_Amount, Job_Card_ID (FK)
PAYMENT	<u>Payment_ID</u> (PK), Payment_Date, Amount, Payment_Mode, Invoice_ID (FK)

7.2 Changes from the Initial Schema

During implementation the initial schema presented earlier was refined as follows:

- INVOICE: the derived attributes Labour_Total and Parts_Total were removed. Both can be calculated from TASK and PART_USAGE, so only Total_Amount is stored as the billed value.
- PAYMENT: Amount_Paid and Payment_Method were renamed to Amount and Payment_Mode, and a CHECK constraint restricts the mode to Cash, Card, UPI or Online.
- PART_USAGE: Quantity_Used was renamed to Quantity.
- Integrity constraints (UNIQUE, CHECK, DEFAULT) were added to every table.

7.3 Normalization

First Normal Form (1NF). Every table has a primary key, every column holds a single atomic value, and there are no repeating groups. For example, a job card with several tasks is not stored in one row; each task is a separate row in TASK linked by Job_Card_ID.

Second Normal Form (2NF). All tables are in 1NF and every primary key is a single column, so no non-key attribute can depend on only part of a key. Hence there are no partial dependencies.

Third Normal Form (3NF). No non-key attribute depends on another non-key attribute. Customer details are kept only in CUSTOMER and not repeated in VEHICLE; the part name and unit price are stored only in SPARE_PART and referenced from PART_USAGE; mechanic details are stored only in MECHANIC and referenced from ASSIGNMENT. This removes update, insertion and deletion anomalies.

Boolean Codd Normal Form (BCNF). In each table the only determinants are candidate keys (for example Customer_ID, Phone and Email in CUSTOMER; Vehicle_ID and Registration_No in VEHICLE), so the schema also satisfies BCNF.

Note on INVOICE.Total_Amount. Total_Amount is a derived value (labour charges plus the price of parts used). It is deliberately retained as a snapshot of the amount billed, so that later changes to a part's Unit_Price do not alter historical invoices.

8. Data Dictionary

The tables below describe the structure of every table, including data types and constraints. PK = Primary Key, FK = Foreign Key.

8.1 CUSTOMER

Stores customer details.

Column	Data Type	Constraint	Description
Customer_ID	INT	PRIMARY KEY	Unique customer identifier
Name	VARCHAR(100)	NOT NULL	Customer name
Phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number
Email	VARCHAR(100)	UNIQUE	E-mail address

8.2 VEHICLE

Vehicles owned by customers.

Column	Data Type	Constraint	Description
Vehicle_ID	INT	PRIMARY KEY	Unique vehicle identifier
Registration_No	VARCHAR(20)	NOT NULL, UNIQUE	Vehicle registration number
Model	VARCHAR(50)	NOT NULL	Vehicle model
Customer_ID	INT	NOT NULL, FK → CUSTOMER	Owner of the vehicle

8.3 SERVICE_BOOKING

Service appointments for vehicles.

Column	Data Type	Constraint	Description
Booking_ID	INT	PRIMARY KEY	Unique booking identifier
Service_Date	DATE	NOT NULL	Scheduled service date
Status	VARCHAR(30)	NOT NULL, DEFAULT 'Booked', CHECK IN (Booked, In Progress, Completed, Cancelled)	Booking status
Vehicle_ID	INT	NOT NULL, FK → VEHICLE	Vehicle being serviced

8.4 JOB_CARD

Work order created for a booking.

Column	Data Type	Constraint	Description
Job_Card_ID	INT	PRIMARY KEY	Unique job card identifier
Created_Date	DATE	NOT NULL	Date the job card was opened
Job_Status	VARCHAR(30)	NOT NULL, DEFAULT 'Open', CHECK IN (Open, In Progress, Completed, Cancelled)	Job progress
Booking_ID	INT	NOT NULL, UNIQUE, FK → SERVICE_BOOKING	Booking this job card belongs to (1:1)

8.5 TASK

Individual tasks within a job card.

Column	Data Type	Constraint	Description
Task_ID	INT	PRIMARY KEY	Unique task identifier
Description	VARCHAR(150)	NOT NULL	Work to be done
Labour_Charge	DECIMAL(10,2)	NOT NULL, CHECK >= 0	Labour cost of the task
Job_Card_ID	INT	NOT NULL, FK → JOB_CARD	Parent job card

8.6 MECHANIC

Mechanics employed at the centre.

Column	Data Type	Constraint	Description
Mechanic_ID	INT	PRIMARY KEY	Unique mechanic identifier
Name	VARCHAR(100)	NOT NULL	Mechanic name
Specialization	VARCHAR(100)	—	Area of expertise
Phone	VARCHAR(15)	UNIQUE	Contact number

8.7 ASSIGNMENT

Allocation of mechanics to tasks (resolves M:N).

Column	Data Type	Constraint	Description
Assignment_ID	INT	PRIMARY KEY	Unique assignment identifier
Start_Time	DATETIME	NOT NULL	Work start time
End_Time	DATETIME	NOT NULL, CHECK End_Time > Start_Time	Work end time
Task_ID	INT	NOT NULL, FK → TASK	Task being performed
Mechanic_ID	INT	NOT NULL, FK → MECHANIC	Mechanic performing the task

8.8 INSPECTION

Inspection findings for a job card.

Column	Data Type	Constraint	Description
Inspection_ID	INT	PRIMARY KEY	Unique inspection identifier
Inspection_Date	DATE	NOT NULL	Date of inspection
Findings	VARCHAR(255)	—	Observations recorded
Job_Card_ID	INT	NOT NULL, FK → JOB_CARD	Related job card

8.9 SPARE_PART

Spare parts inventory.

Column	Data Type	Constraint	Description
Part_ID	INT	PRIMARY KEY	Unique part identifier
Part_Name	VARCHAR(100)	NOT NULL	Name of the part
Stock_Quantity	INT	NOT NULL, DEFAULT 0, CHECK ≥ 0	Units in stock
Unit_Price	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0	Price per unit

8.10 PART_USAGE

Parts consumed by a job card.

Column	Data Type	Constraint	Description
Usage_ID	INT	PRIMARY KEY	Unique usage record identifier
Quantity	INT	NOT NULL, CHECK > 0	Units of the part used
Job_Card_ID	INT	NOT NULL, FK $\rightarrow$ JOB_CARD	Job card using the part
Part_ID	INT	NOT NULL, FK $\rightarrow$ SPARE_PART	Part used

8.11 INVOICE

Bill generated for a job card.

Column	Data Type	Constraint	Description
Invoice_ID	INT	PRIMARY KEY	Unique invoice identifier
Invoice_Date	DATE	NOT NULL	Date of billing
Total_Amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK ≥ 0	Total payable (labour + parts)
Job_Card_ID	INT	NOT NULL, UNIQUE, FK $\rightarrow$ JOB_CARD	Job card billed (1:1)

8.12 PAYMENT

Payments received against invoices.

Column	Data Type	Constraint	Description
Payment_ID	INT	PRIMARY KEY	Unique payment identifier
Payment_Date	DATE	NOT NULL	Date of payment
Amount	DECIMAL(10,2)	NOT NULL, CHECK > 0	Amount paid
Payment_Mode	VARCHAR(30)	NOT NULL, CHECK IN (Cash, Card, UPI, Online)	Mode of payment
Invoice_ID	INT	NOT NULL, FK $\rightarrow$ INVOICE	Invoice being paid

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Database Creation (DDL)

```
CREATE DATABASE VehicleServiceCentre;
USE VehicleServiceCentre;
```

9.2 Table Creation (DDL)

The twelve tables were created in dependency order so that every referenced (parent) table exists before the table that refers to it.

1. CUSTOMER

```
CREATE TABLE CUSTOMER (
    Customer_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15) NOT NULL UNIQUE,
    Email VARCHAR(100) UNIQUE
);
```

2. VEHICLE

```
CREATE TABLE VEHICLE (
    Vehicle_ID INT PRIMARY KEY,
    Registration_No VARCHAR(20) NOT NULL UNIQUE,
    Model VARCHAR(50) NOT NULL,
    Customer_ID INT NOT NULL,
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID)
);
```

3. SERVICE_BOOKING

```
CREATE TABLE SERVICE_BOOKING (
    Booking_ID INT PRIMARY KEY,
    Service_Date DATE NOT NULL,
    Status VARCHAR(30) NOT NULL DEFAULT 'Booked',
    Vehicle_ID INT NOT NULL,
    FOREIGN KEY (Vehicle_ID) REFERENCES VEHICLE(Vehicle_ID),
    CHECK (Status IN ('Booked', 'In Progress', 'Completed', 'Cancelled'))
);
```

4. JOB_CARD

```
CREATE TABLE JOB_CARD (
    Job_Card_ID INT PRIMARY KEY,
    Created_Date DATE NOT NULL,
    Job_Status VARCHAR(30) NOT NULL DEFAULT 'Open',
    Booking_ID INT NOT NULL UNIQUE,
    FOREIGN KEY (Booking_ID) REFERENCES SERVICE_BOOKING(Booking_ID),
    CHECK (Job_Status IN ('Open', 'In Progress', 'Completed', 'Cancelled'))
);
```

5. TASK

```
CREATE TABLE TASK (
    Task_ID INT PRIMARY KEY,
    Description VARCHAR(150) NOT NULL,
    Labour_Charge DECIMAL(10,2) NOT NULL,
    Job_Card_ID INT NOT NULL,
    FOREIGN KEY (Job_Card_ID) REFERENCES JOB_CARD(Job_Card_ID),
    CHECK (Labour_Charge >= 0)
);
```

6. MECHANIC

```
CREATE TABLE MECHANIC (
    Mechanic_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Specialization VARCHAR(100),
    Phone VARCHAR(15) UNIQUE
);
```

7. ASSIGNMENT

```
CREATE TABLE ASSIGNMENT (
    Assignment_ID INT PRIMARY KEY,
    Start_Time DATETIME NOT NULL,
    End_Time DATETIME NOT NULL,
    Task_ID INT NOT NULL,
    Mechanic_ID INT NOT NULL,
    FOREIGN KEY (Task_ID) REFERENCES TASK(Task_ID),
    FOREIGN KEY (Mechanic_ID) REFERENCES MECHANIC(Mechanic_ID),
    CHECK (End_Time > Start_Time)
);
```

8. INSPECTION

```
CREATE TABLE INSPECTION (
    Inspection_ID INT PRIMARY KEY,
    Inspection_Date DATE NOT NULL,
    Findings VARCHAR(255),
    Job_Card_ID INT NOT NULL,
    FOREIGN KEY (Job_Card_ID) REFERENCES JOB_CARD(Job_Card_ID)
);
```

9. SPARE_PART

```
CREATE TABLE SPARE_PART (
    Part_ID INT PRIMARY KEY,
    Part_Name VARCHAR(100) NOT NULL,
    Stock_Quantity INT NOT NULL DEFAULT 0,
    Unit_Price DECIMAL(10,2) NOT NULL,
    CHECK (Stock_Quantity >= 0),
    CHECK (Unit_Price >= 0)
);
```

10. PART_USAGE

```
CREATE TABLE PART_USAGE (
    Usage_ID INT PRIMARY KEY,
    Quantity INT NOT NULL,
    Job_Card_ID INT NOT NULL,
    Part_ID INT NOT NULL,
    FOREIGN KEY (Job_Card_ID) REFERENCES JOB_CARD(Job_Card_ID),
    FOREIGN KEY (Part_ID) REFERENCES SPARE_PART(Part_ID),
    CHECK (Quantity > 0)
);
```

11. INVOICE

```
CREATE TABLE INVOICE (
    Invoice_ID INT PRIMARY KEY,
    Invoice_Date DATE NOT NULL,
    Total_Amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    Job_Card_ID INT NOT NULL UNIQUE,
    FOREIGN KEY (Job_Card_ID) REFERENCES JOB_CARD(Job_Card_ID),
    CHECK (Total_Amount >= 0)
);
```

12. PAYMENT

```
CREATE TABLE PAYMENT (
    Payment_ID INT PRIMARY KEY,
    Payment_Date DATE NOT NULL,
    Amount DECIMAL(10,2) NOT NULL,
    Payment_Mode VARCHAR(30) NOT NULL,
    Invoice_ID INT NOT NULL,
    FOREIGN KEY (Invoice_ID) REFERENCES INVOICE(Invoice_ID),
    CHECK (Amount > 0),
    CHECK (Payment_Mode IN ('Cash', 'Card', 'UPI', 'Online'))
);
```

9.3 Data Insertion (DML)

Sample data was inserted into every table using INSERT statements. The first rows of each insert are shown below; the complete script is available in the GitHub repository (Appendix).

CUSTOMER

```
INSERT INTO CUSTOMER VALUES
(1, 'Ravi', '9876543210', 'ravi@gmail.com'),
(2, 'Anu', '9876543211', 'anu@gmail.com'),
(3, 'Arun', '9876543212', 'arun@gmail.com');
-- ... 7 more rows
```

VEHICLE

```
INSERT INTO VEHICLE VALUES
(101, 'TS09AB1234', 'Creta', 1),
(102, 'TS10CD5678', 'Swift', 2),
(103, 'TS08EF9012', 'City', 3);
-- ... 7 more rows
```

SERVICE_BOOKING

```
INSERT INTO SERVICE_BOOKING VALUES
(201, '2026-09-10', 'Completed', 101),
(202, '2026-09-12', 'In Progress', 102),
(203, '2026-09-14', 'Booked', 103);
-- ... 7 more rows
```

JOB_CARD

```
INSERT INTO JOB_CARD VALUES
(301, '2026-09-10', 'Completed', 201),
(302, '2026-09-12', 'In Progress', 202),
(303, '2026-09-14', 'Open', 203);
-- ... 7 more rows
```

TASK

```
INSERT INTO TASK VALUES
(401, 'Oil Change', 800.00, 301),
(402, 'Brake Check', 500.00, 301),
(403, 'AC Service', 1200.00, 302);
-- ... 9 more rows
```

MECHANIC

```
INSERT INTO MECHANIC VALUES
(501, 'Ram', 'Engine', '9000000001'),
(502, 'Raj', 'Brakes', '9000000002'),
(503, 'Sam', 'AC', '9000000003');
-- ... 5 more rows
```

ASSIGNMENT

```
INSERT INTO ASSIGNMENT VALUES
(601, '2026-09-10 09:00:00', '2026-09-10 10:00:00', 401, 501),
(602, '2026-09-10 10:15:00', '2026-09-10 11:00:00', 402, 502),
(603, '2026-09-12 09:30:00', '2026-09-12 11:00:00', 403, 503);
-- ... 9 more rows
```

INSPECTION

```
INSERT INTO INSPECTION VALUES
(701, '2026-09-10', 'Oil checked', 301),
(702, '2026-09-12', 'AC needs service', 302),
(703, '2026-09-14', 'Alignment needed', 303);
-- ... 7 more rows
```

SPARE_PART

```
INSERT INTO SPARE_PART VALUES
(801, 'Oil Filter', 20, 450.00),
(802, 'Brake Pad', 10, 1800.00),
(803, 'Air Filter', 15, 700.00);
-- ... 7 more rows
```

PART_USAGE

```
INSERT INTO PART_USAGE VALUES
(901, 1, 301, 801),
(902, 1, 301, 803),
(903, 1, 302, 807);
-- ... 9 more rows
```

INVOICE

```
INSERT INTO INVOICE VALUES
(1001, '2026-09-10', 2450.00, 301),
(1002, '2026-09-08', 4700.00, 304),
(1003, '2026-09-11', 1950.00, 309);
-- ... 1 more rows
```

PAYMENT

```
INSERT INTO PAYMENT VALUES
(1101, '2026-09-10', 2450.00, 'UPI', 1001),
(1102, '2026-09-08', 4700.00, 'Card', 1002),
(1103, '2026-09-11', 1950.00, 'Cash', 1003);
-- ... 1 more rows
```

9.4 Verifying the Tables

```
SHOW TABLES;
```

```
+-----+
| Tables_in_VehicleServiceCentre |
+-----+
| ASSIGNMENT                      |
| CUSTOMER                       |
| INSPECTION                     |
| INVOICE                        |
| JOB_CARD                       |
| MECHANIC                       |
| PART_USAGE                     |
| PAYMENT                        |
| SERVICE_BOOKING                |
| SPARE_PART                     |
| TASK                           |
| VEHICLE                        |
+-----+
```

9.5 Sample Outputs (SELECT)

SELECT * FROM CUSTOMER;

```
+-----+-----+-----+-----+
| Customer_ID | Name  | Phone | Email |
+-----+-----+-----+-----+
| 1 | Ravi | 9876543210 | ravi@gmail.com |
| 2 | Anu | 9876543211 | anu@gmail.com |
| 3 | Arun | 9876543212 | arun@gmail.com |
| 4 | Priya | 9876543213 | priya@gmail.com |
| 5 | Kiran | 9876543214 | kiran@gmail.com |
| 6 | Sneha | 9876543215 | sneha@gmail.com |
| 7 | Rahul | 9876543216 | rahul@gmail.com |
| 8 | Meena | 9876543217 | meena@gmail.com |
| 9 | Vijay | 9876543218 | vijay@gmail.com |
| 10 | Pooja | 9876543219 | pooja@gmail.com |
+-----+-----+-----+-----+
```

SELECT * FROM VEHICLE;

```
+-----+-----+-----+-----+
| Vehicle_ID | Registration_No | Model | Customer_ID |
+-----+-----+-----+-----+
| 101 | TS09AB1234 | Creta | 1 |
| 102 | TS10CD5678 | Swift | 2 |
| 103 | TS08EF9012 | City | 3 |
| 104 | TS11GH3456 | Nexon | 4 |
| 105 | TS12IJ7890 | Glanza | 5 |
| 106 | TS13KL2468 | Seltos | 6 |
| 107 | TS14MN1357 | i20 | 7 |
| 108 | TS15OP8642 | Amaze | 8 |
| 109 | TS16QR9753 | Harrier | 9 |
| 110 | TS17ST6420 | Baleno | 10 |
+-----+-----+-----+-----+
```

SELECT * FROM SERVICE_BOOKING;

Booking_ID	Service_Date	Status	Vehicle_ID
201	2026-09-10	Completed	101
202	2026-09-12	In Progress	102
203	2026-09-14	Booked	103
204	2026-09-08	Completed	104
205	2026-09-15	In Progress	105
206	2026-09-16	Booked	106
207	2026-09-17	Booked	107
208	2026-09-18	Booked	108
209	2026-09-11	Completed	109
210	2026-09-13	Completed	110

SELECT * FROM JOB_CARD;

Job_Card_ID	Created_Date	Job_Status	Booking_ID
301	2026-09-10	Completed	201
302	2026-09-12	In Progress	202
303	2026-09-14	Open	203
304	2026-09-08	Completed	204
305	2026-09-15	In Progress	205
306	2026-09-16	Open	206
307	2026-09-17	Open	207
308	2026-09-18	Open	208
309	2026-09-11	Completed	209
310	2026-09-13	Completed	210

SELECT * FROM TASK;

Task_ID	Description	Labour_Charge	Job_Card_ID
401	Oil Change	800.00	301
402	Brake Check	500.00	301
403	AC Service	1200.00	302
404	Wheel Alignment	600.00	303
405	Engine Service	1500.00	304
406	Battery Check	300.00	305
407	Brake Repair	1000.00	306
408	General Check	400.00	307
409	Filter Change	500.00	308
410	Engine Tune	1300.00	309
411	Tyre Check	450.00	310
412	AC Check	700.00	310

SELECT * FROM MECHANIC;

Mechanic_ID	Name	Specialization	Phone
501	Ram	Engine	9000000001
502	Raj	Brakes	9000000002
503	Sam	AC	9000000003
504	Ajay	General	9000000004
505	Amit	Electrical	9000000005
506	Ravi	Engine	9000000006
507	Kiran	Brakes	9000000007
508	Vijay	General	9000000008

SELECT * FROM ASSIGNMENT;

Assignment_ID	Start_Time	End_Time	Task_ID	Mechanic_ID
601	2026-09-10 09:00:00	2026-09-10 10:00:00	401	501
602	2026-09-10 10:15:00	2026-09-10 11:00:00	402	502
603	2026-09-12 09:30:00	2026-09-12 11:00:00	403	503
604	2026-09-14 10:00:00	2026-09-14 11:00:00	404	504
605	2026-09-08 09:00:00	2026-09-08 11:00:00	405	506
606	2026-09-15 09:00:00	2026-09-15 10:00:00	406	505
607	2026-09-16 09:00:00	2026-09-16 10:30:00	407	507
608	2026-09-17 09:00:00	2026-09-17 10:00:00	408	508
609	2026-09-18 09:30:00	2026-09-18 10:30:00	409	504
610	2026-09-11 09:00:00	2026-09-11 10:30:00	410	501
611	2026-09-13 10:00:00	2026-09-13 11:00:00	411	508
612	2026-09-13 11:15:00	2026-09-13 12:00:00	412	503

SELECT * FROM INSPECTION;

Inspection_ID	Inspection_Date	Findings	Job_Card_ID
701	2026-09-10	Oil checked	301
702	2026-09-12	AC needs service	302
703	2026-09-14	Alignment needed	303
704	2026-09-08	Engine normal	304
705	2026-09-15	Battery low	305
706	2026-09-16	Brake pads worn	306
707	2026-09-17	Vehicle checked	307
708	2026-09-18	Filter needs change	308
709	2026-09-11	Engine needs tune	309
710	2026-09-13	Tyres checked	310

SELECT * FROM SPARE_PART;

Part_ID	Part_Name	Stock_Quantity	Unit_Price
801	Oil Filter	20	450.00
802	Brake Pad	10	1800.00
803	Air Filter	15	700.00
804	Spark Plug	25	350.00
805	Battery	8	4500.00
806	Engine Oil	30	650.00
807	AC Filter	12	900.00
808	Brake Fluid	18	500.00
809	Coolant	20	400.00
810	Drive Belt	10	1200.00

SELECT * FROM PART_USAGE;

Usage_ID	Quantity	Job_Card_ID	Part_ID
901	1	301	801
902	1	301	803
903	1	302	807
904	1	304	802
905	4	304	804
906	1	305	805
907	1	306	802
908	1	307	806
909	1	308	803
910	1	309	806
911	1	310	809
912	1	310	807

SELECT * FROM INVOICE;

Invoice_ID	Invoice_Date	Total_Amount	Job_Card_ID
1001	2026-09-10	2450.00	301
1002	2026-09-08	4700.00	304
1003	2026-09-11	1950.00	309
1004	2026-09-13	2650.00	310

SELECT * FROM PAYMENT;

Payment_ID	Payment_Date	Amount	Payment_Mode	Invoice_ID
1101	2026-09-10	2450.00	UPI	1001
1102	2026-09-08	4700.00	Card	1002
1103	2026-09-11	1950.00	Cash	1003
1104	2026-09-13	2650.00	Online	1004

10. Queries with Outputs

The following queries were executed on the populated database. Queries 1–3 demonstrate selection and joins, Queries 4–6 demonstrate aggregate functions, Query 7 demonstrates grouping, Query 8 joins three tables, and Queries 9–10 produce financial summaries.

Query 1: Display all completed job cards

```
SELECT * FROM JOB_CARD
WHERE Job_Status = 'Completed';
```

Output:

Job_Card_ID	Created_Date	Job_Status	Booking_ID
301	2026-09-10	Completed	201
304	2026-09-08	Completed	204
309	2026-09-11	Completed	209
310	2026-09-13	Completed	210

Result: Four job cards (301, 304, 309, 310) have been completed.

Query 2: Customer and vehicle details

```
SELECT c.Name, v.Registration_No, v.Model
FROM CUSTOMER c
JOIN VEHICLE v
ON c.Customer_ID = v.Customer_ID;
```

Output:

Name	Registration_No	Model
Ravi	TS09AB1234	Creta
Anu	TS10CD5678	Swift
Arun	TS08EF9012	City
Priya	TS11GH3456	Nexon
Kiran	TS12IJ7890	Glanza
Sneha	TS13KL2468	Seltos
Rahul	TS14MN1357	i20
Meena	TS15OP8642	Amaze
Vijay	TS16QR9753	Harrier
Pooja	TS17ST6420	Baleno

Result: Each customer is listed with the registered vehicle they own.

Query 3: Complete service details (4-table join)

```
SELECT c.Name, v.Registration_No, v.Model,
       sb.Service_Date, jc.Job_Status
FROM CUSTOMER c
JOIN VEHICLE v
ON c.Customer_ID = v.Customer_ID
JOIN SERVICE_BOOKING sb
ON v.Vehicle_ID = sb.Vehicle_ID
JOIN JOB_CARD jc
ON sb.Booking_ID = jc.Booking_ID;
```

Output:

Name	Registration_No	Model	Service_Date	Job_Status
Ravi	TS09AB1234	Creta	2026-09-10	Completed
Anu	TS10CD5678	Swift	2026-09-12	In Progress
Arun	TS08EF9012	City	2026-09-14	Open
Priya	TS11GH3456	Nexon	2026-09-08	Completed
Kiran	TS12IJ7890	Glanza	2026-09-15	In Progress
Sneha	TS13KL2468	Seltos	2026-09-16	Open
Rahul	TS14MN1357	i20	2026-09-17	Open
Meena	TS15OP8642	Amaze	2026-09-18	Open
Vijay	TS16QR9753	Harrier	2026-09-11	Completed
Pooja	TS17ST6420	Baleno	2026-09-13	Completed

Result: Traces the full path Customer → Vehicle → Booking → Job Card.

Query 4: Total number of customers

```
SELECT COUNT(*) AS Total_Customers
FROM CUSTOMER;
```

Output:

Total_Customers
10

Result: The centre currently has 10 registered customers.

Query 5: Highest labour charge

```
SELECT MAX(Labour_Charge) AS Highest_Labour_Charge
FROM TASK;
```

Output:

Highest_Labour_Charge
1500.00

Result: The most expensive task is the Engine Service at ₹1500.00.

Query 6: Average labour charge

```
SELECT AVG(Labour_Charge) AS Average_Labour_Charge
FROM TASK;
```

Output:

Average_Labour_Charge
770.833333

Result: Average labour charge across the 12 tasks is about ₹770.83.

Query 7: Mechanic workload

```
SELECT m.Name, COUNT(a.Assignment_ID) AS Total_Assignments
FROM MECHANIC m
JOIN ASSIGNMENT a
ON m.Mechanic_ID = a.Mechanic_ID
GROUP BY m.Mechanic_ID, m.Name;
```

Output:

Name	Total_Assignments
Ram	2
Raj	1
Sam	2
Ajay	2
Amit	1
Ravi	1
Kiran	1
Vijay	2

Result: Ram, Sam, Ajay and Vijay each handled two assignments; the others handled one.

Query 8: Spare parts used per job card

```
SELECT jc.Job_Card_ID, sp.Part_Name, pu.Quantity
FROM PART_USAGE pu
JOIN JOB_CARD jc
ON pu.Job_Card_ID = jc.Job_Card_ID
JOIN SPARE_PART sp
ON pu.Part_ID = sp.Part_ID;
```

Output:

Job_Card_ID	Part_Name	Quantity
301	Oil Filter	1
301	Air Filter	1
302	AC Filter	1
304	Brake Pad	1
304	Spark Plug	4
305	Battery	1
306	Brake Pad	1
307	Engine Oil	1
308	Air Filter	1
309	Engine Oil	1
310	Coolant	1
310	AC Filter	1

Result: Lists every part consumed with its quantity, e.g. four spark plugs on job card 304.

Query 9: Total revenue

```
SELECT SUM(Total_Amount) AS Total_Revenue
FROM INVOICE;
```

Output:

Total_Revenue
11750.00

Result: Total billed revenue across all invoices is ₹11,750.00.

Query 10: Payments by mode

```
SELECT Payment_Mode, COUNT(*) AS Number_of_Payments
FROM PAYMENT
GROUP BY Payment_Mode;
```

Output:

Payment_Mode	Number_of_Payments
Card	1
Cash	1
Online	1
UPI	1

Result: One payment each was received by Card, Cash, Online and UPI.

Query 11: Query given during Presentation-II

```
USE VehicleServiceCentre;

SELECT c.Name, v.Model AS Car_Name,
       b.Status AS Booking_Status
FROM CUSTOMER c
JOIN VEHICLE v ON c.Customer_ID = v.Customer_ID
JOIN SERVICE_BOOKING b ON v.Vehicle_ID = b.Vehicle_ID
WHERE c.Name = 'Ravi'
      AND b.Status = 'Completed';
```

Output:

Name	Car_Name	Booking_Status
Ravi	Creta	Completed

1 row in set (0.016 sec)

Result: The query joins CUSTOMER, VEHICLE and SERVICE_BOOKING to show that customer Ravi's car, a Creta, has a completed service booking.

11. UI Design and Screenshots

The system includes a desktop user interface developed with Python's Tkinter library and connected to the MySQL database through mysql-connector-python. The user first connects to the database, then selects any of the twelve tables from a drop-down list to browse its records, insert a new record using a form, or delete a selected record. The screenshots below document the working application.

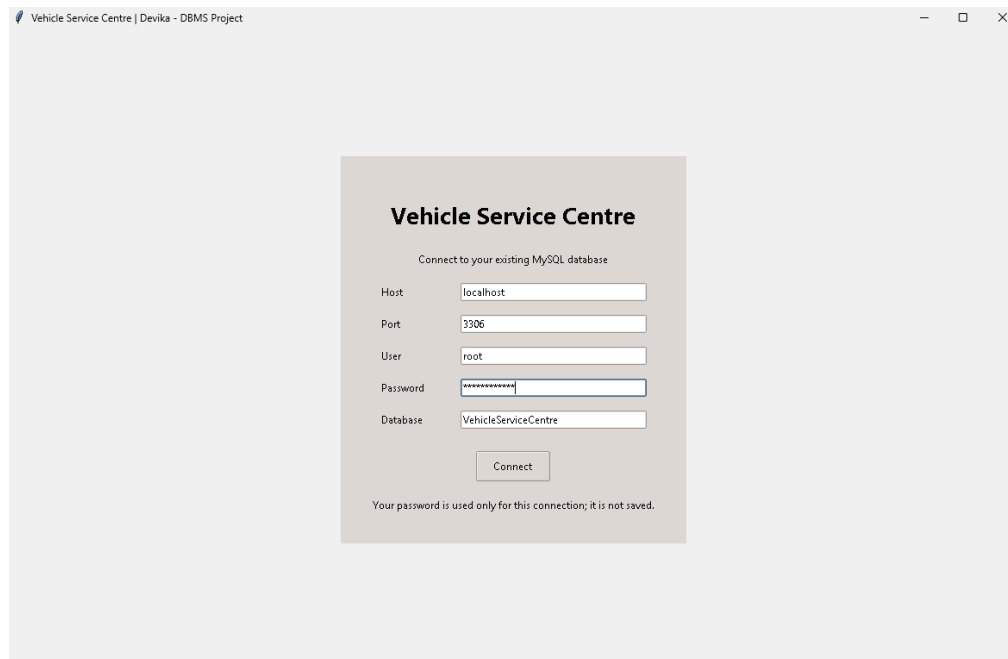


Figure 11.1: User Interface (database connection screen)

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **CUSTOMER** Refresh records Delete selected record

CUSTOMER | 10 records | Connected to MySQL

Customer ID	Name	Phone	Email
1	Ravi	9876543210	ravi@gmail.com
2	Anu	9876543211	anu@gmail.com
3	Arun	9876543212	arun@gmail.com
4	Priya	9876543213	priya@gmail.com
5	Kiran	9876543214	kiran@gmail.com
6	Sneha	9876543215	sneha@gmail.com
7	Rahul	9876543216	rahul@gmail.com
8	Meena	9876543217	meena@gmail.com
9	Vijay	9876543218	vijay@gmail.com
10	Pooja	9876543219	pooja@gmail.com

Insert a new record

Customer ID * Name * Phone * Email (optional)

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.2: Customer Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **VEHICLE** Refresh records Delete selected record

VEHICLE | 10 records | Connected to MySQL

Vehicle ID	Registration No	Model	Customer ID
101	TS09AB1234	Creta	1
102	TS10CD5678	Swift	2
103	TS08EF9012	City	3
104	TS11GH3456	Nexon	4
105	TS12IJ7890	Glanza	5
106	TS13KL2468	Seltos	6
107	TS14MN1357	i20	7
108	TS15OP8642	Amaze	8
109	TS16QR9753	Harrier	9
110	TS17ST6420	Baleno	10

Insert a new record

Vehicle ID * Registration No * Model * Customer ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.3: Vehicle Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **TASK** Refresh records Delete selected record

TASK | 12 records | Connected to MySQL

Task ID	Description	Labour Charge	Job Card ID
401	Oil Change	800.00	301
402	Brake Check	500.00	301
403	AC Service	1200.00	302
404	Wheel Alignment	600.00	303
405	Engine Service	1500.00	304
406	Battery Check	300.00	305
407	Brake Repair	1000.00	306
408	General Check	400.00	307
409	Filter Change	500.00	308
410	Engine Tune	1300.00	309
411	Tyre Check	450.00	310
412	AC Check	700.00	310

Insert a new record

Task ID * Description * Labour Charge * Job Card ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.4: Task Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **JOB_CARD** Refresh records Delete selected record

JOB_CARD | 10 records | Connected to MySQL

Job Card ID	Created Date	Job Status	Booking ID
301	2026-09-10	Completed	201
302	2026-09-12	In Progress	202
303	2026-09-14	Open	203
304	2026-09-08	Completed	204
305	2026-09-15	In Progress	205
306	2026-09-16	Open	206
307	2026-09-17	Open	207
308	2026-09-18	Open	208
309	2026-09-11	Completed	209
310	2026-09-13	Completed	210

Insert a new record

Job Card ID * Created Date * Job Status * Booking ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.5: Job Card Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **SERVICE_BOOKING** Refresh records

Disconnect Delete selected record

SERVICE_BOOKING | 11 records | Connected to MySQL

Booking ID	Service Date	Status	Vehicle ID
201	2026-09-10	Completed	101
202	2026-09-12	In Progress	102
203	2026-09-14	Booked	103
204	2026-09-08	Completed	104
205	2026-09-15	In Progress	105
206	2026-09-16	Booked	106
207	2026-09-17	Booked	107
208	2026-09-18	Booked	108
209	2026-09-11	Completed	109
210	2026-09-13	Completed	110
2999	2026-10-06	Booked	101

Insert a new record

Booking ID * Service Date * Status * Vehicle ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.6: Service Booking Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **MECHANIC** Refresh records

Disconnect Delete selected record

MECHANIC | 8 records | Connected to MySQL

Mechanic ID	Name	Specialization	Phone
501	Ram	Engine	9000000001
502	Raj	Brakes	9000000002
503	Sam	AC	9000000003
504	Ajay	General	9000000004
505	Amit	Electrical	9000000005
506	Ravi	Engine	9000000006
507	Kiran	Brakes	9000000007
508	Vijay	General	9000000008

Insert a new record

Mechanic ID * Name * Specialization (optional) Phone (optional)

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.7: Mechanic Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **ASSIGNMENT** Refresh records Delete selected record

ASSIGNMENT | 12 records | Connected to MySQL

Assignment ID	Start Time	End Time	Task ID	Mechanic ID
601	2026-09-10 09:00:00	2026-09-10 10:00:00	401	501
602	2026-09-10 10:15:00	2026-09-10 11:00:00	402	502
603	2026-09-12 09:30:00	2026-09-12 11:00:00	403	503
604	2026-09-14 10:00:00	2026-09-14 11:00:00	404	504
605	2026-09-08 09:00:00	2026-09-08 11:00:00	405	506
606	2026-09-15 09:00:00	2026-09-15 10:00:00	406	505
607	2026-09-16 09:00:00	2026-09-16 10:30:00	407	507
608	2026-09-17 09:00:00	2026-09-17 10:00:00	408	508
609	2026-09-18 09:30:00	2026-09-18 10:30:00	409	504
610	2026-09-11 09:00:00	2026-09-11 10:30:00	410	501
611	2026-09-13 10:00:00	2026-09-13 11:00:00	411	508
612	2026-09-13 11:15:00	2026-09-13 12:00:00	412	503

Insert a new record

Assignment ID * Start Time * End Time * Task ID *

Mechanic ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.8: Assignment Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **INSPECTION** Refresh records Delete selected record

INSPECTION | 10 records | Connected to MySQL

Inspection ID	Inspection Date	Findings	Job Card ID
701	2026-09-10	Oil checked	301
702	2026-09-12	AC needs service	302
703	2026-09-14	Alignment needed	303
704	2026-09-08	Engine normal	304
705	2026-09-15	Battery low	305
706	2026-09-16	Brake pads worn	306
707	2026-09-17	Vehicle checked	307
708	2026-09-18	Filter needs change	308
709	2026-09-11	Engine needs tune	309
710	2026-09-13	Tyres checked	310

Insert a new record

Inspection ID * Inspection Date * Findings (optional) Job Card ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.9: Inspection Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **SPARE_PART** Refresh records Disconnect Delete selected record

SPARE_PART | 10 records | Connected to MySQL

Part ID	Part Name	Stock Quantity	Unit Price
801	Oil Filter	20	450.00
802	Brake Pad	10	1800.00
803	Air Filter	15	700.00
804	Spark Plug	25	350.00
805	Battery	8	4500.00
806	Engine Oil	30	650.00
807	AC Filter	12	900.00
808	Brake Fluid	18	500.00
809	Coolant	20	400.00
810	Drive Belt	10	1200.00

Insert a new record

Part ID * Part Name * Stock Quantity * Unit Price *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.10: Spare Part Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **PART_USAGE** Refresh records Disconnect Delete selected record

PART_USAGE | 12 records | Connected to MySQL

Usage ID	Quantity	Job Card ID	Part ID
901	1	301	801
902	1	301	803
903	1	302	807
904	1	304	802
905	4	304	804
906	1	305	805
907	1	306	802
908	1	307	806
909	1	308	803
910	1	309	806
911	1	310	809
912	1	310	807

Insert a new record

Usage ID * Quantity * Job Card ID * Part ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.11: Part Usage Table

Vehicle Service Centre | Devika - DBMS Project

Vehicle Service Centre

Select table: **INVOICE** Refresh records

Disconnect

Delete selected record

INVOICE | 4 records | Connected to MySQL

Invoice ID	Invoice Date	Total Amount	Job Card ID
1001	2026-09-10	2450.00	301
1002	2026-09-08	4700.00	304
1003	2026-09-11	1950.00	309
1004	2026-09-13	2650.00	310

Insert a new record

Invoice ID * Invoice Date * Total Amount * Job Card ID *

Insert record Dates: YYYY-MM-DD | Times: YYYY-MM-DD HH:MM:SS | IDs are entered manually

Live MySQL connection | Select a row to delete | Refresh to see changes made in the SQL client

Figure 11.12: Invoice Table

12. Implementation Details

12.1 Technology Stack

Layer	Technology
Database	MySQL 8.0 (InnoDB storage engine)
Query language	SQL – DDL, DML and DQL
Application language	Python 3.x
User interface	Tkinter (desktop GUI)
Database connector	mysql-connector-python
SQL client	MySQL Workbench / MySQL Command Line Client
Design tools	Canva for ER diagram and slides
Repository	GitHub

12.2 Database Connectivity

The application opens a connection screen in which the user enters the host (localhost), port (3306), user name, password and database name (VehicleServiceCentre) and presses Connect. The connection is made with mysql-connector-python, and the password is used only for that session and is not saved. After connecting, the status bar shows the selected table, its number of records and the message “Connected to MySQL”. The Disconnect button closes the connection. Referential integrity between tables is enforced by the MySQL server using InnoDB foreign keys, so invalid operations are rejected by the database and the error is displayed in the interface.

12.3 Application Features

- View records of all 12 tables by selecting a table from a drop-down list, with a Refresh button to reload the data.
- Insert a new record using a form generated for the selected table; mandatory fields are marked with an asterisk (*) and optional fields are labelled optional.
- Select related records using drop-down lists for foreign keys (for example Customer ID in VEHICLE or Job Card ID in INVOICE), which prevents entering a non-existent parent record.
- Default values for date and status fields (for example today’s date and the status Booked or Open).
- Delete a selected record, with a confirmation prompt before the deletion is carried out.
- Input validation and display of database error messages when a constraint is violated.

12.4 Key Code Snippets

(a) A child table with keys and constraints – ASSIGNMENT. The table links a task to a mechanic and uses a CHECK constraint to prevent an end time earlier than the start time.

```
CREATE TABLE ASSIGNMENT (
    Assignment_ID INT PRIMARY KEY,
    Start_Time DATETIME NOT NULL,
    End_Time DATETIME NOT NULL,
    Task_ID INT NOT NULL,
    Mechanic_ID INT NOT NULL,
    FOREIGN KEY (Task_ID) REFERENCES TASK(Task_ID),
    FOREIGN KEY (Mechanic_ID) REFERENCES MECHANIC(Mechanic_ID),
    CHECK (End_Time > Start_Time)
);
```

(b) Status validation – SERVICE_BOOKING. A DEFAULT value and a CHECK constraint ensure that a booking can only hold one of four valid statuses.

```
CREATE TABLE SERVICE_BOOKING (
    Booking_ID INT PRIMARY KEY,
    Service_Date DATE NOT NULL,
    Status VARCHAR(30) NOT NULL DEFAULT 'Booked',
    Vehicle_ID INT NOT NULL,
    FOREIGN KEY (Vehicle_ID) REFERENCES VEHICLE(Vehicle_ID),
    CHECK (Status IN ('Booked', 'In Progress', 'Completed', 'Cancelled'))
);
```

(c) Multi-table join – complete service details. Four tables are joined through their foreign keys to trace a service from the customer to the job card.

```
SELECT c.Name, v.Registration_No, v.Model,
       sb.Service_Date, jc.Job_Status
FROM CUSTOMER c
JOIN VEHICLE v
ON c.Customer_ID = v.Customer_ID
JOIN SERVICE_BOOKING sb
ON v.Vehicle_ID = sb.Vehicle_ID
JOIN JOB_CARD jc
ON sb.Booking_ID = jc.Booking_ID;
```

(d) Aggregation with grouping – mechanic workload.

```
SELECT m.Name, COUNT(a.Assignment_ID) AS Total_Assignments
FROM MECHANIC m
JOIN ASSIGNMENT a
ON m.Mechanic_ID = a.Mechanic_ID
GROUP BY m.Mechanic_ID, m.Name;
```

13. Testing

Testing was carried out in two parts. Constraint testing (T1–T12) attempted invalid inserts and deletes to confirm that the database rejects them with the correct error, and query testing (T13–T16) compared query results with the values expected from the sample data. All tests were executed on the final database.

ID	Test Case	Input	Expected Result	Actual Result	Status
T1	Duplicate phone number in CUSTOMER	INSERT customer with Phone 9876543210	Rejected (UNIQUE)	ERROR 1062: Duplicate entry for key Phone	Pass
T2	NULL in mandatory column	INSERT customer with Name = NULL	Rejected (NOT NULL)	ERROR 1048: Column Name cannot be null	Pass
T3	Vehicle for non-existent customer	INSERT vehicle with Customer_ID = 99	Rejected (FOREIGN KEY)	ERROR 1452: foreign key constraint fails	Pass
T4	Invalid booking status	INSERT booking with Status = 'Pending'	Rejected (CHECK)	ERROR 4025: CHECK constraint failed	Pass
T5	Negative labour charge	INSERT task with Labour_Charge = -50	Rejected (CHECK)	ERROR 4025: CHECK constraint failed	Pass
T6	Zero payment amount	INSERT payment with Amount = 0	Rejected (CHECK)	ERROR 4025: CHECK constraint failed	Pass
T7	Invalid payment mode	INSERT payment with Mode = 'Cheque'	Rejected (CHECK)	ERROR 4025: CHECK constraint failed	Pass
T8	End time before start time	INSERT assignment, End_Time < Start_Time	Rejected (CHECK)	ERROR 4025: CHECK constraint failed	Pass
T9	Second job card for same booking	INSERT job card with Booking_ID = 201	Rejected (UNIQUE)	ERROR 1062: Duplicate entry for key Booking_ID	Pass
T10	Delete customer who owns a vehicle	DELETE customer with Customer_ID = 1	Rejected (referential integrity)	ERROR 1451: cannot delete parent row	Pass
T11	Negative stock quantity	INSERT spare part with Stock_Quantity = -1	Rejected (CHECK)	ERROR 4025: CHECK constraint failed	Pass
T12	Default value for booking status	INSERT booking without Status	Status = 'Booked'	Row inserted, Status = Booked	Pass
T13	Query 1 – completed jobs	SELECT ... WHERE Job_Status = 'Completed'	4 rows	4 rows (301, 304, 309, 310)	Pass
T14	Query 4 – customer count	SELECT COUNT(*) FROM CUSTOMER	10	10	Pass
T15	Query 9 – total revenue	SELECT SUM(Total_Amount) FROM INVOICE	11750.00	11750.00	Pass
T16	Query 10 – payments by mode	GROUP BY Payment_Mode	4 groups, 1 each	Card 1, Cash 1, Online 1, UPI 1	Pass

All 16 test cases passed. The results confirm that primary keys, unique constraints, NOT NULL, CHECK constraints and foreign keys protect the database against invalid data, and that the queries return correct results.

14. Conclusion and Future Enhancements

14.1 Conclusion

This project successfully designed and implemented a relational database for a Vehicle Service Centre. Starting from an ER diagram, a twelve-table schema in Third Normal Form was created and implemented in MySQL with primary keys, foreign keys and CHECK, UNIQUE, NOT NULL and DEFAULT constraints. The system links customers, vehicles, bookings, job cards, tasks, mechanics, parts, invoices and payments into one consistent structure, and the ten SQL queries demonstrate that useful information such as pending jobs, mechanic workload, parts consumption and revenue can be obtained easily. Constraint testing showed that invalid data is reliably rejected.

14.2 Future Enhancements

- Extend the desktop interface with record editing, search and reports, and develop a web or mobile front-end so that staff and customers can use the system from anywhere.
- Use triggers to reduce SPARE_PART.Stock_Quantity automatically when parts are used, and to calculate INVOICE.Total_Amount from labour and parts.
- Create views and stored procedures for daily revenue, pending jobs and low-stock reports.
- Add user roles and authentication (admin, receptionist, mechanic) with access control.
- Send SMS or e-mail reminders for service due dates and ready vehicles.
- Integrate an online payment gateway and support partial payments with outstanding-balance tracking.
- Add a supplier and purchase-order module for stock replenishment.
- Add indexes on frequently searched columns and maintain regular backups.

15. References

- [1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.
- [2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.
- [3] R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed., McGraw-Hill, 2003.
- [4] Oracle Corporation, MySQL 8.0 Reference Manual. Available:
<https://dev.mysql.com/doc/refman/8.0/en/>
- [5] P. P. Chen, “The Entity-Relationship Model – Toward a Unified View of Data,” ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9–36, 1976.

16. Appendix: GitHub Repository Link

The complete SQL script (database creation, table creation, sample data and queries), the ER diagram and this report are available in the project repository:

<https://github.com/DevikaKatralla/DBMS-Course-Project>

Repository contents:

- VehicleServiceCentre.sql – DDL, DML and queries
- ER diagram and relational schema
- Project presentation and report
- Screenshots